- PRÁCTICA 2: CAMBIO DE LA FRECUENCIA DE MUESTREO

PRÁCTICA 2: CAMBIO DE LA FRECUENCIA DE MUESTREO

▶ DIEZMADO POR UN FACTOR ENTERO

El diezmado de una señal consiste en modificar la frecuencia original de muestreo ($fs1$) reduciéndola por un factor entero de M, de tal forma que: $fs2$ = $fs1$ /M Lo que se traduce en tomar una de cada M muestras de la señal de entrada, e ignorar el resto. A partir de la señal de audio facilitada, realice los siguientes apartados, respondiendo a las preguntas que se plantean: A partir de la señal de audio facilitada, realice los siguientes apartados, respondiendo a las preguntas que se plantean:

a) Lea el archivo que le ha facilitado el profesor con una señal de audio. La señal obtenida se denominará x[n]. Reproduzca esta señal de audio. Indique la frecuencia de muestreo fs1fs1.

```
[x, fs1] = audioread("PDS_P2_3B_LE2_G4.wav");
sound(x,fs1);
disp(fs1);
```

b) Represente en el tiempo x(t), a partir de las muestras x[n], con el vector de tiempo empezando en t = 0.

```
% x, fs1 ya cargados
N = size(x,1);
t = (0:N-1)'/fs1;          % vector tiempo empezando en 0 (columnas)

% Representación
figure; plot(t, x);
xlabel('t (s)'); ylabel('Amplitude');
title('Representación temporal x_n'); grid on;
```

c) Calcule la señal y[n] como la parte de x[n] que contiene sonido. Es decir, ignore de x[n] las muestras de los intervalos de reposo (|x[n]| < 0,01 V) 1 anteriores y posteriores a la voz grabada.

```
% x, fs1 ya cargados
% Umbral en voltios (según enunciado)
thr = 0.01;

% Si x tiene dos canales, usar la envolvente (max absoluto entre canales)
if size(x,2) > 1
    env = max(abs(x), [], 2);
else
    env = abs(x);
end

% Encontrar índices activos (|x| >= thr)
active = find(env >= thr);

if isempty(active)
    error('No se encuentran muestras por encima del umbral.');
end

% Primer y último índice con sonido
i0 = active(1);
i1 = active(end);

% Extraer y[n]
y = x(i0:i1, :);   % conserva canales si existen
n_y = (0:size(y,1)-1)';  % índice n empezando en 0 para y[n]
t_y = n_y / fs1;         % vector tiempo empezando en t=0

% PLOT
figure;
plot(t_y, y); xlabel('t (s)'); ylabel('Amplitude'); title('y_n');
```

d) Genere una función de Matlab en la que se diezme una señal muestreada, por un factor M dado. Indique los argumentos de entrada y salida de la función generada.

```
function y_n = Diezmador(x_n, M)
    y_n = x_n(1:M:end);
end
```

e) Realice el diezmado de y[n] con un coeficiente M=2 . La señal resultante será g[n].

```
% y es la señal extraída anteriormente (y[n])
M = 2;
g = Diezmador(y, M);      % g es g[n] decimada
fs_g = fs1 / M;           % nueva frecuencia de muestreo

% PLOT de la señal diezmada g[n]
```

```
figure;
plot((0:length(g)-1)'/fs_g, g);
xlabel('t (s)'); ylabel('Amplitude'); title('g[n] - Señal Decimada');
grid on;
% x y fs ya cargados (o usar g y fs_g si decimaste)
if size(x,2) > 1
    sig = x(:,1);   % seleccionar primer canal (o usar mean(x,2))
else
    sig = x;
end

N = length(sig);
X = fft(sig);           % transformada
Xsh = fftshift(X);      % desplaza 0 al centro

% vector de frecuencia centrado en 0
f = (-floor(N/2) : ceil(N/2)-1) * (fs1 / N);

% magnitud (lineal). Para dB usa 20*log10(...)
mag = abs(Xsh) / N;

figure;
plot(f, mag);
xlabel('Frecuencia (Hz)');
ylabel('Magnitud');
title('Espectro centrado g_n');
grid on;
xlim([-fs1/2 fs1/2]);
```

f) Calcule la nueva frecuencia de muestreo ($fs2$) de la señal y[n], con la que da lugar a g[n]. ¿Cuál será la nueva frecuencia de muestreo de x[n]?

```
fs2 = fs1 / M
%PLOT en frecuencia de y_n y g_n

% seleccionar primer canal si hay más de uno
if size(y,2) > 1, y = y(:,1); else y = y; end
if size(g_n,2) > 1, g = g_n(:,1); else g = g_n; end

% espectro de y_n
Ny = length(y);
Ysh = fftshift(fft(y));
fy = (-floor(Ny/2) : ceil(Ny/2)-1) * (fs1 / Ny);
Py_db = 20*log10( abs(Ysh)/Ny + eps );

% espectro de g_n
Ng = length(g);
Gsh = fftshift(fft(g));
fg = (-floor(Ng/2) : ceil(Ng/2)-1) * (fs2 / Ng);
Pg_db = 20*log10( abs(Gsh)/Ng + eps );

% plot en la misma figura
figure;
plot(fy, Py_db, 'b-', 'LineWidth', 1.1); hold on;
```

```
plot(fg, Pg_db, 'r-', 'LineWidth', 1.1);
hold off;
xlabel('Frecuencia (Hz)');
ylabel('Magnitud (dB)');
title('Espectros centrados de y[n] (azul) y g[n] (rojo)');
legend('y[n]', 'g[n]');
grid on;
xlim([ -max(fs1,fs2)/2 , max(fs1,fs2)/2 ]);
```

g) Justificación en el dominio del tiempo

Comparación visual de las muestras originales y diezmadas para verificar la coincidencia de valores.

```
tg = (0:length(g)-1).' / fs2_y;
figure;
plot(ty, y, 'b-o'); hold on;                    % Original en azul con círculos
plot(tg, g, 'r*');                              % Diezmada en rojo con asteriscos
xlim([0.5, 0.505]);                             % Zoom para apreciar las muestras
xlabel('Tiempo (s)'); ylabel('Amplitud');
title('Justificación Tiempo: y[n] vs g[n]');
legend('Original y[n]', 'Diezmada g[n]');
grid on;
```

Análisis: La gráfica muestra que el diezmado es correcto ya que las muestras de g[n] (asteriscos rojos) coinciden exactamente con una de cada dos muestras de y[n] (círculos azules). Al corregir el eje de tiempos con fs2, la forma de la onda se mantiene idéntica.

h) Justificación en el dominio de la frecuencia.

Análisis espectral utilizando frecuencias analógicas (Hz) para verificar la ausencia de distorsión.

```
Ny = length(y); Ng = length(g);
Y_mag = abs(fft(y))/Ny;
G_mag = abs(fft(g))/Ng;
fy = (0:Ny-1)*(fs1/Ny);
fg = (0:Ng-1)*(fs2_y/Ng);

figure;
plot(fy(1:floor(Ny/2)), Y_mag(1:floor(Ny/2)),'b-o'); hold on;
plot(fg(1:floor(Ng/2)), G_mag(1:floor(Ng/2)),'r*');
xlim([1000, 1050]);                             % Zoom en un rango de frecuencias
xlabel('Frecuencia (Hz)'); ylabel('Magnitud');
title('Justificación Frecuencia: y[n] vs g[n]');
```

```
legend('Original y[n]', 'Diezmada g[n]');
grid on;
```

Análisis: El espectro de g[n] coincide con el de y[n] dentro de la nueva banda de paso (0 a fs2/2 Hz). La coincidencia de los marcadores demuestra que la magnitud de las componentes se mantiene constante. El diezmado es válido porque la señal no tenía energía significativa por encima de la nueva frecuencia de Nyquist, evitando el solapamiento.

i) Análisis de Aliasing.

Pregunta: **¿Qué pasaría al diezmar y[n] si su espectro tuviese energía entre fs1/2 y fs2/2? ¿Qué habría que hacer para evitarlo?**

Respuesta: Si la señal tuviese energía en ese rango, se produciría el fenómeno de **aliasing (solapamiento espectral). Las componentes de alta frecuencia se "doblarían" sobre la banda de interés, distorsionando la señal resultante.**

Para evitarlo, es necesario aplicar un filtro digital paso bajo (antialiasing) antes del proceso de diezmado. Este filtro debe tener una frecuencia de corte de fc = fs2/2 para asegurar que se elimine toda la energía que queda fuera de la nueva franja de Nyquist antes de descartar las muestras.

▶ INTERPOLACIÓN POR UN FACTOR ENTERO

Con la interpolación, modificamos la frecuencia original de muestreo incrementándolaa por un factor entero L. Partiendo de la señal y[n] del bloque anterior, se realizan los siguientes apartados:

a) Genera una función de Matlab en la que se interpole una señal muestreada por un factor L dado. Indique los argumentos de entrada y salida de la función generada.

```
function y_l = interpolar(x, L)

% Generar función de Matlab para interpolar dado un facto L

% x = la entrada que se va a interpolar
% L = factor de interpolación
% y_l = salida de la función, señal interpolada

% Generamos la nueva frecuencia de muestreo, partiendo de la Fs obtenida
% con nuestra señal de audio, la cual es fs = 96000
```

```
% En primer lugar, descartamos aquellas L que no sean válidas:

if ((L<1) || (floor(L) ~= L)) % Verifico que L sea entero y >=1
    error('El factor L debe ser un entero mayor o igual a 1')
end

% Garantizado que la L con la que trabajamos  es válida...

% Un interpolador inserta ceros cada L muestras, ahora hemos de añadir

y_l = zeros(1, L*length(x));
y_l(1:L:end) = x;


end
```

b) Realice la interpolación de y[n] con un coeficiente L = 2. La señal resultante será h[n].

- Siendo la primera parte la de la obtención de y[n]:

```
% >>>>> Teniendo lo realizado anteriormente para llegar a y[n]:

[x, fs1] = audioread("PDS_P2_3B_LE2_G4.wav");
sound(x,fs1);
disp(fs1);

% x, fs1 ya cargados
N = size(x,1);
t = (0:N-1)'/fs1;          % vector tiempo empezando en 0 (columnas)

% Representación
figure; plot(t, x);
xlabel('t (s)'); ylabel('Amplitude');
title('Representación temporal x_n'); grid on;

% x, fs1 ya cargados
thr = 0.01; % Umbral en voltios (según enunciado)

% Si x tiene dos canales, usar la envolvente (max absoluto entre canales)
if size(x,2) > 1
    env = max(abs(x), [], 2);
else
    env = abs(x);
end

% Encontrar índices activos (|x| >= thr)
active = find(env >= thr);

if isempty(active)
    error('No se encuentran muestras por encima del umbral.');
end
```

```
% Primer y último índice con sonido
i0 = active(1);
i1 = active(end);

% Extraer y[n]
y = x(i0:i1, :);   % conserva canales si existen
n_y = (0:size(y,1)-1)';  % índice n empezando en 0 para y[n]
t_y = n_y / fs1;         % vector tiempo empezando en t=0
```

- Una vez obtenida y[n], interpolamos con L=2:

```
% >>>>> Teniendo la señal y[n]
L = 2;
h_n = interpolar(y, L); % Obtengo la nueva señal interpolada
```

c) Calcule la nueva frecuencia de muestreo de la señal y[n], con la que da lugar a h[n].

Teniendo en cuenta que la señal y[n] es un fragmento de x[n] (aquel que nos interesaba y que cumplía con el umbral marcado), la frecuencia de muestreo es la misma tanto para y[n] como para x[n].

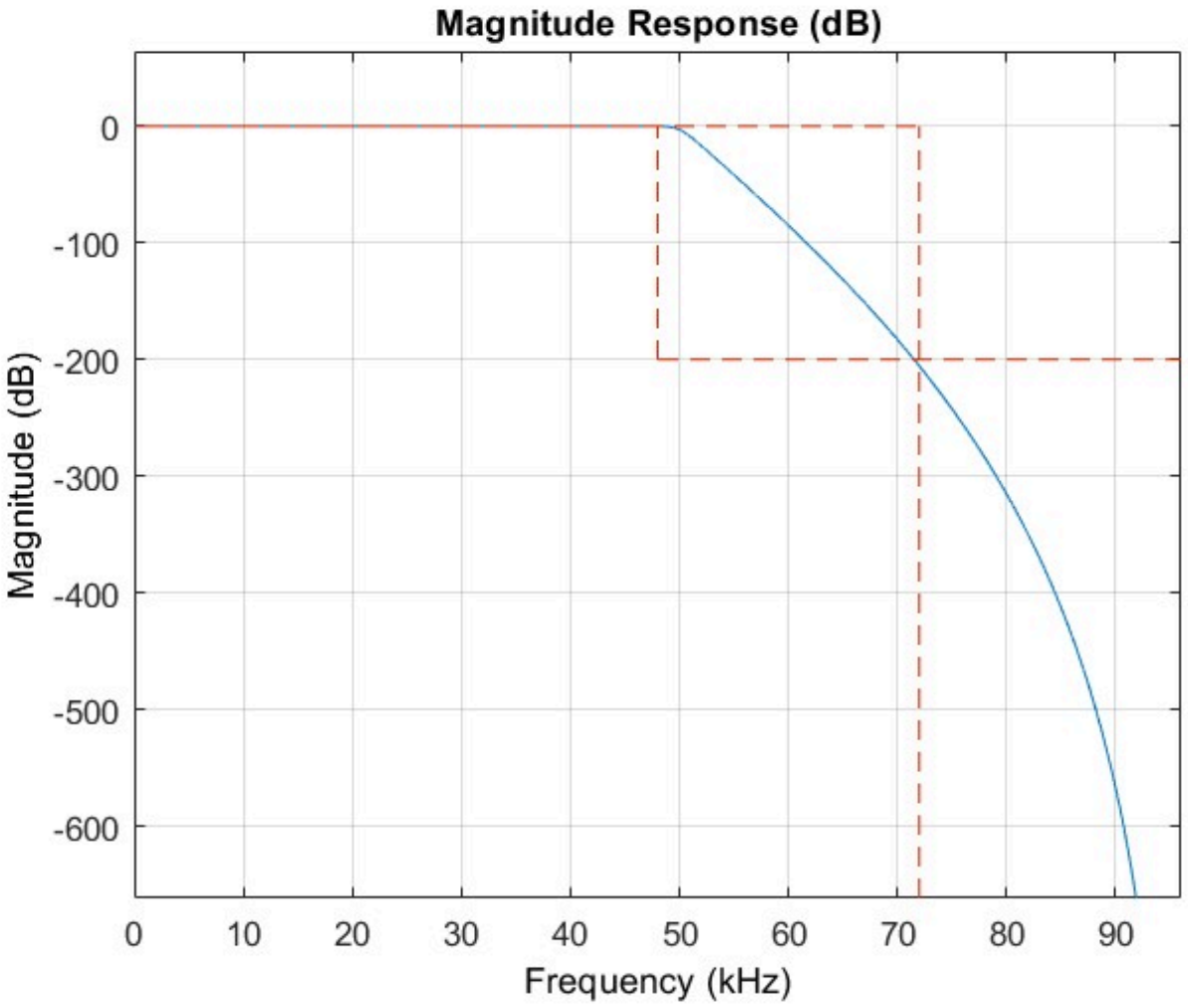
d) Filtre la señal h[n] con el filtro facilitado por el profesor. La señal resultante será k[n].

Hemos de tener en cuenta que los argumentos de entrada del filtro son:

- Señal muestreada a filtrar.
- Frecuencia de muestreo de dicha señal.
- Ganancia del filtro en la banda de paso.
- Frecuencia de corte analógica del filtro.

```
% Filtramos la señal h[n] con el filtro que nos han dado, obetniendo k:
k = Filtro(h_n, fs1*L, L, fs1/2); % fs1/2 es la de corte en analógico
% Lo que tenemos que cuidar es el eje X, el de amplitudes no es tan
% importante
```

Obteniendo la siguiente imagen donde se muestra la caída del filtro, el cual se puede observar que no es ideal pues su caída no es abrupta:



e) Indique cuánto debe valer la ganancia del filtro en la banda de paso.

La ganancia del filtro en la banda de paso debe ser L para restaurar la energía que se perdió al insertar ceros, visible en cómo la amplitud de cada réplica tras interpolar baja a 1/L. Al insertar ceros, no pierdes energía total (pues esta se conserva), pero sí "densidad de energía", porque ahora la señal ocupa más muestras; será el filtro el encargado de compensar eso multiplicando por L.

f) Justifique la frecuencia de corte del filtro empleado.

La frecuencia de corte tiene que ser del 1/(2*L) para evitar que las réplicas (originalmente centradas en 1 en frecuencia digital) se cuelen en la franja de Nyquist al comprimir el espectro 1/L. Hemos de seguir respetando la periodicidad entre +-1/2.

g) Justifique gráficamente, en el dominio del tiempo, que los procesos de interpolación y filtrado realizados son correctos. Utilice marcadores en las muestras para apreciar las diferencias más fácilmente.

- Interpolación

En primer lugar, hemos de observar cómo las dimensiones de nuestros vectores cambian al interpolar (añadir ceros). Como nuestro factor de interpolación es L = 2, tendremos L-1 = 1 cero entre cada muestra de la señal original. Hemos de verificar que h[n], que es la señal interpolada, tiene el doble de dimensión que y[n]:

```
disp(length(y));
disp(length(h_n));
```

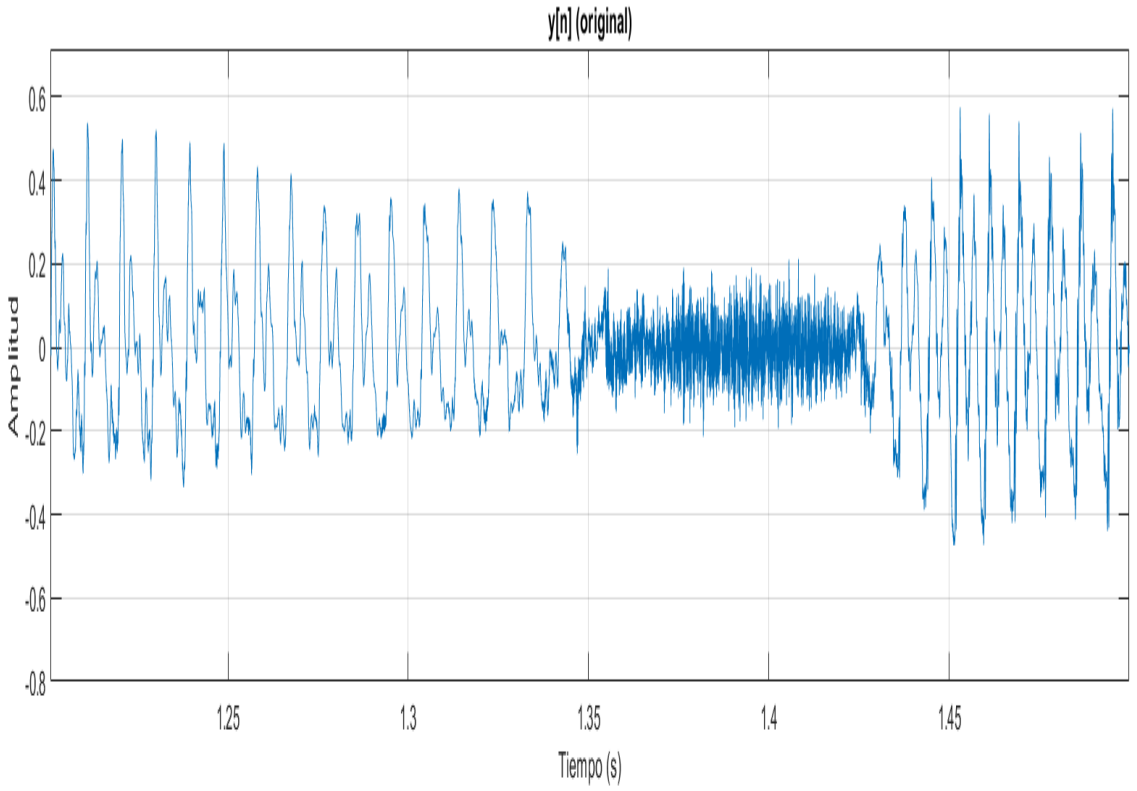
Quedando comprobado que sí se duplica la longitud de mi señal en tiempo:

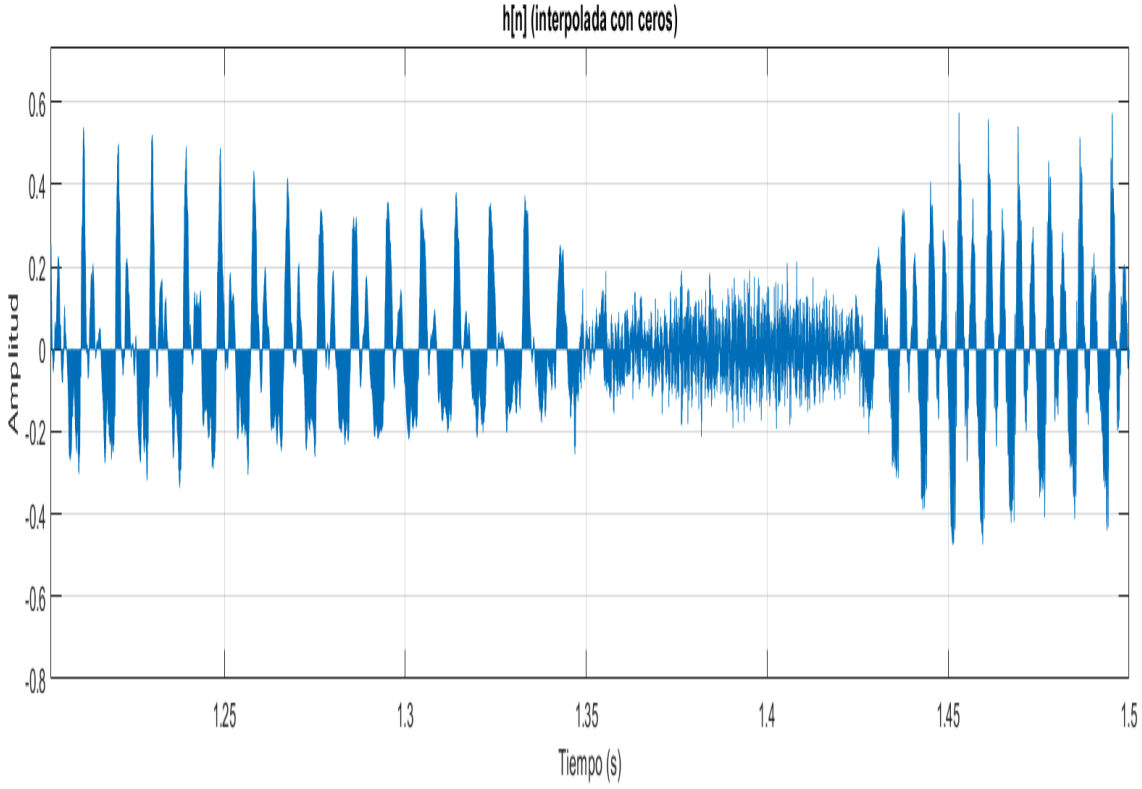
```
275286

550572
```

Ahora pasamos a la representación gráfica pedida:

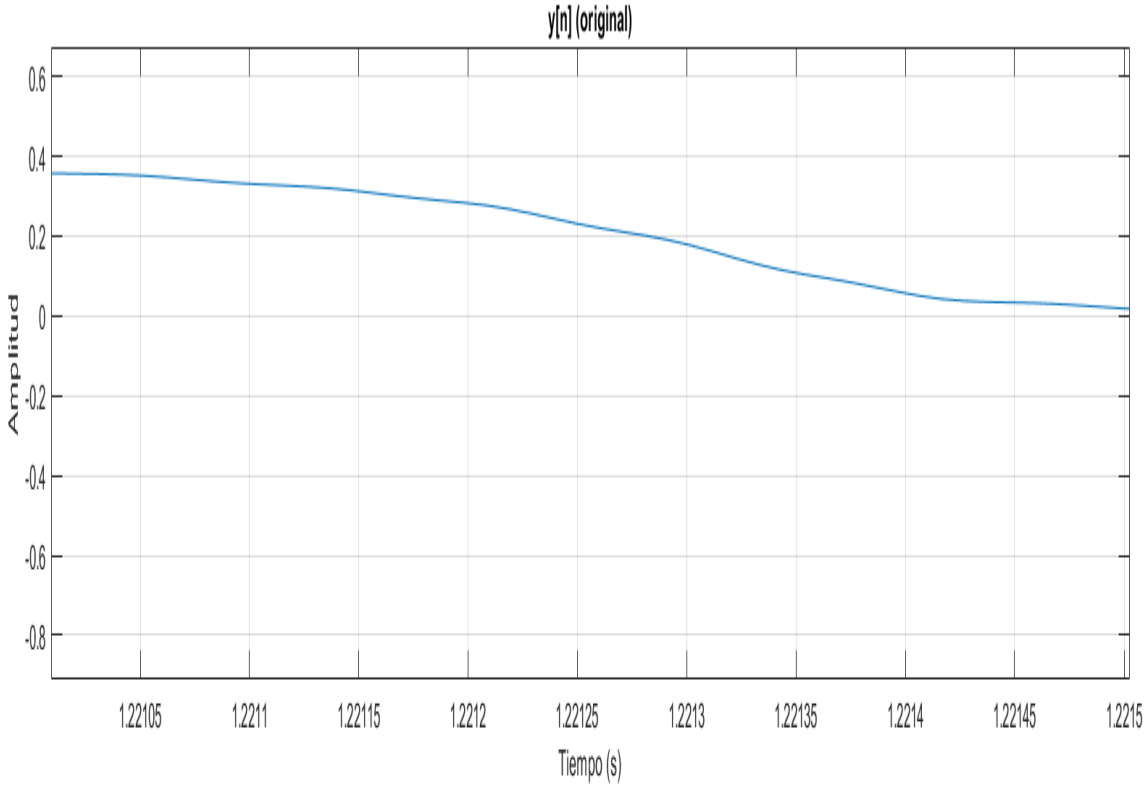
```
% Longitudes
Nh = length(h_n); % Señal interpolada antes del filtrado
%disp(Nh)
Nk = length(k); % Señal interpolada tras el filtrado
%disp(Nk)

% Vector de tiempo (ambas tienen la misma fs2)
th = (0:Nh-1)'/(fs1*L); % Señal interpolada antes del filtrado
tk = (0:Nk-1)'/(fs1*L); % Señal interpolada tras el filtrado

% Para ver antes del filtrado al añadir la interpolación:
figure;
subplot(2,1,1);
plot(t_y, y);
xlabel('Tiempo (s)');
ylabel('Amplitud');
title('y[n] (original)');
grid on;

subplot(2,1,2);
plot(th, h_n);
xlabel('Tiempo (s)');
ylabel('Amplitud');
title('h[n] (interpolada con ceros)');
grid on;
```

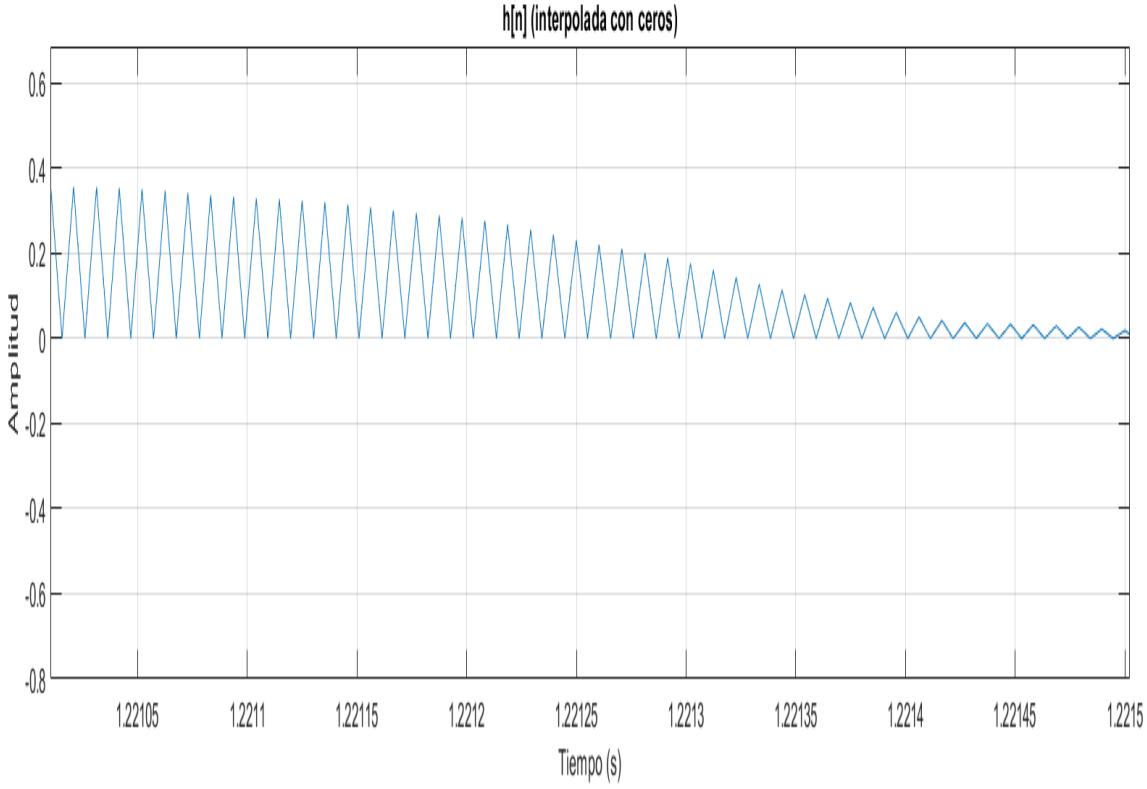
A simple vista, parece que nuestra señal interpolada está rellena y no se aprecian los ceros a simple vista:





Sin embargo, al hacer zoom podemos observar como sí hay saltos bruscos entre muestras causados por haber introducido ceros entre muestras, lo que prueba que la interpolación por ceros está bien hecha:





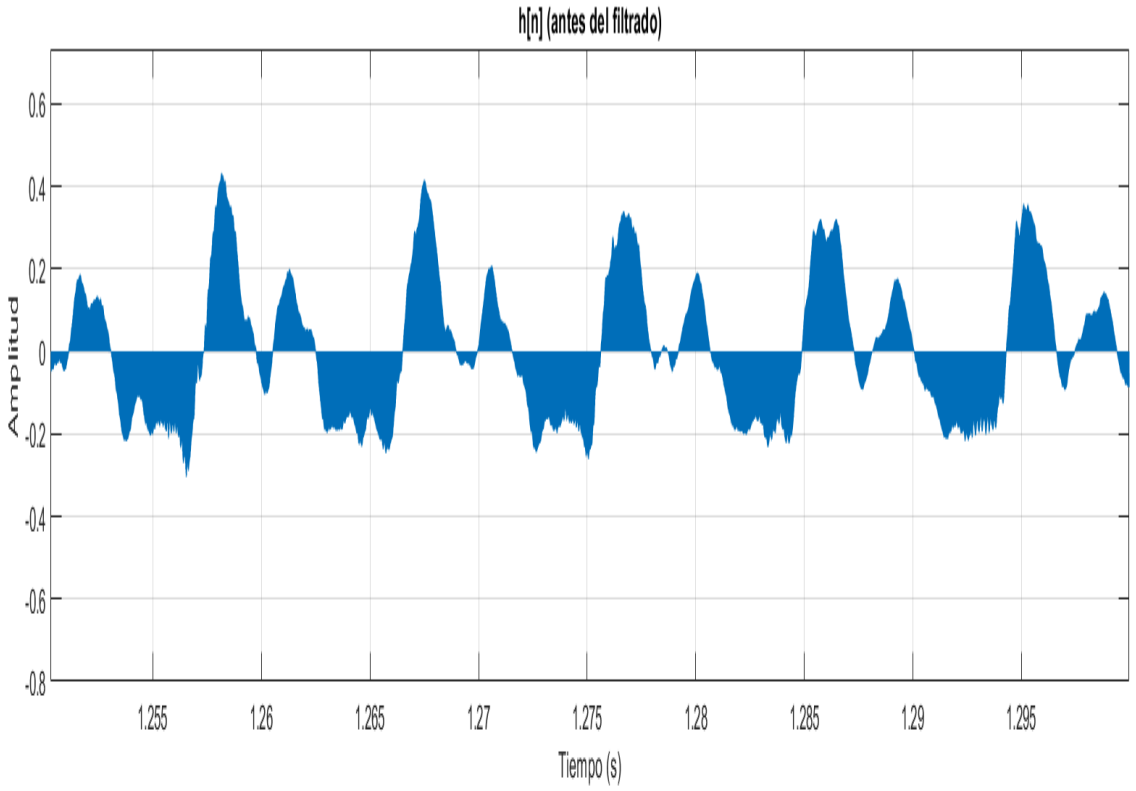
- Filtrado

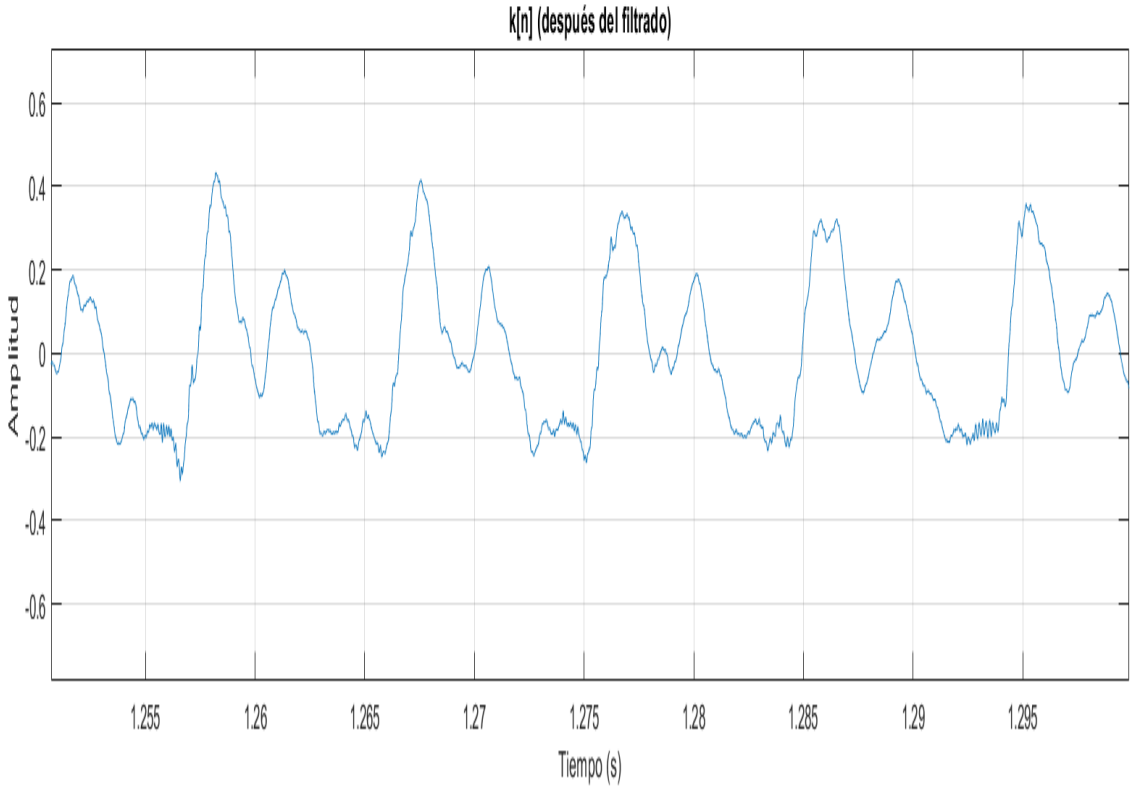
El filtrado nos permitirá observar cómo partimos de una señal con cambios bruscos provocados por los ceros, llegando a una señal suave y continua, con una forma similar a la de la original.

En este caso, **las dimensiones** de ambas señales, tanto antes como después del filtrado **son las mismas**.

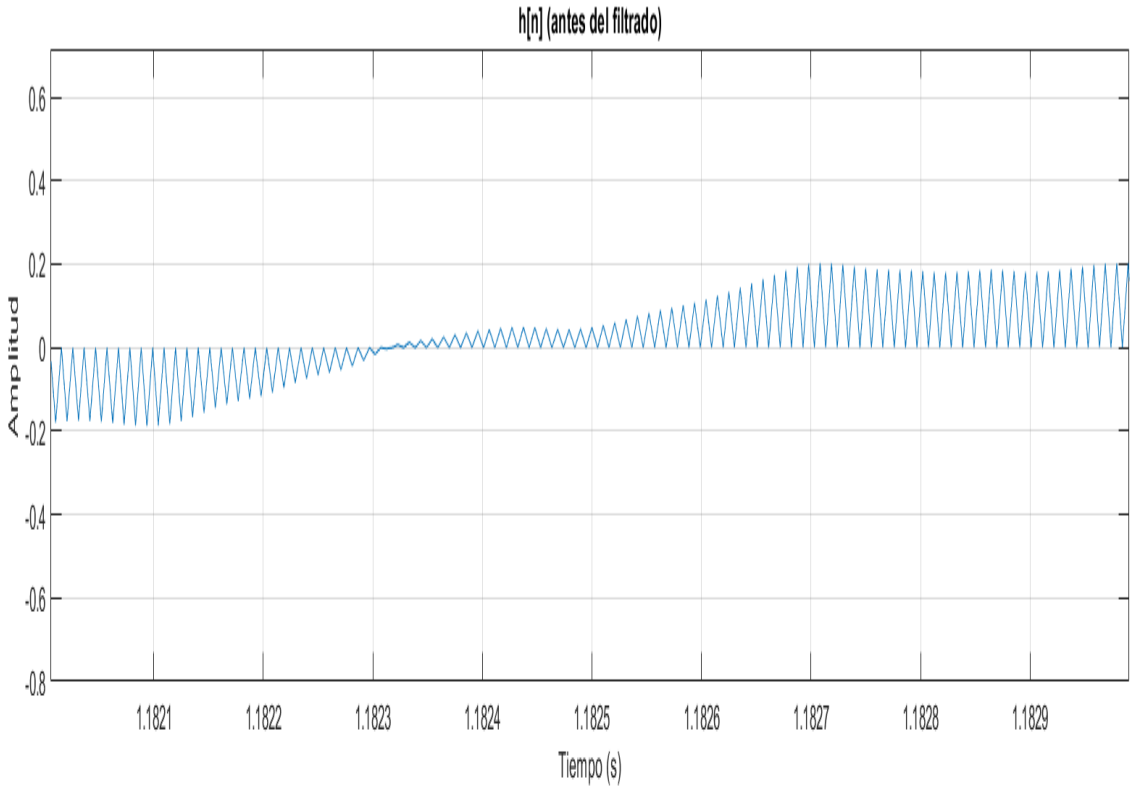
```
% Para ver el efecto del filtro:
figure;
subplot(2,1,1);
plot(th, h_n);
xlabel('Tiempo (s)');
ylabel('Amplitud');
title('h[n] (antes del filtrado)');
grid on;

subplot(2,1,2);
plot(tk, k);
xlabel('Tiempo (s)');
ylabel('Amplitud');
title('k[n] (después del filtrado)');
grid on;
```

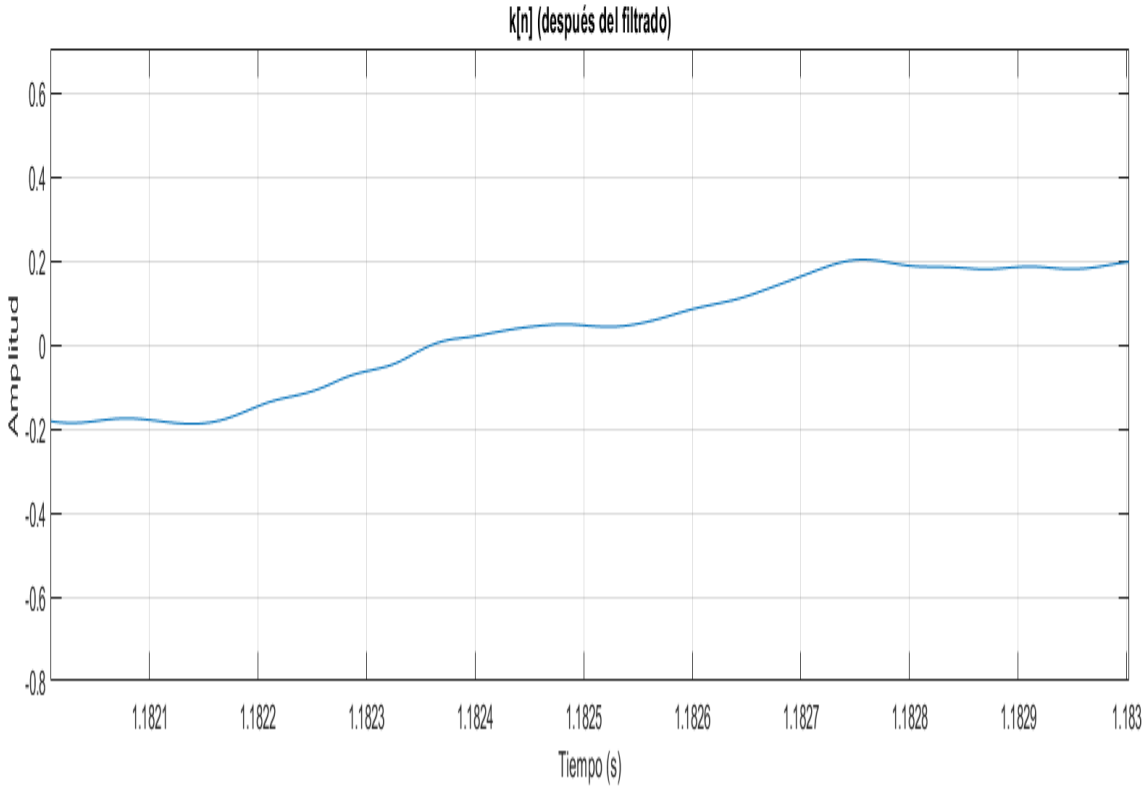
En un primer vistazo, puede parecer que se ha "deshecho" la operación anterior, pues parece que pasamos de una señal rellena o otra hueca:





Sin embargo, veamos cómo haciendo zoom logramos ver dicha señal suavizada, la cual surge de suavizar los ceros sin por ello modificar las dimensiones de mi señal interpolada:

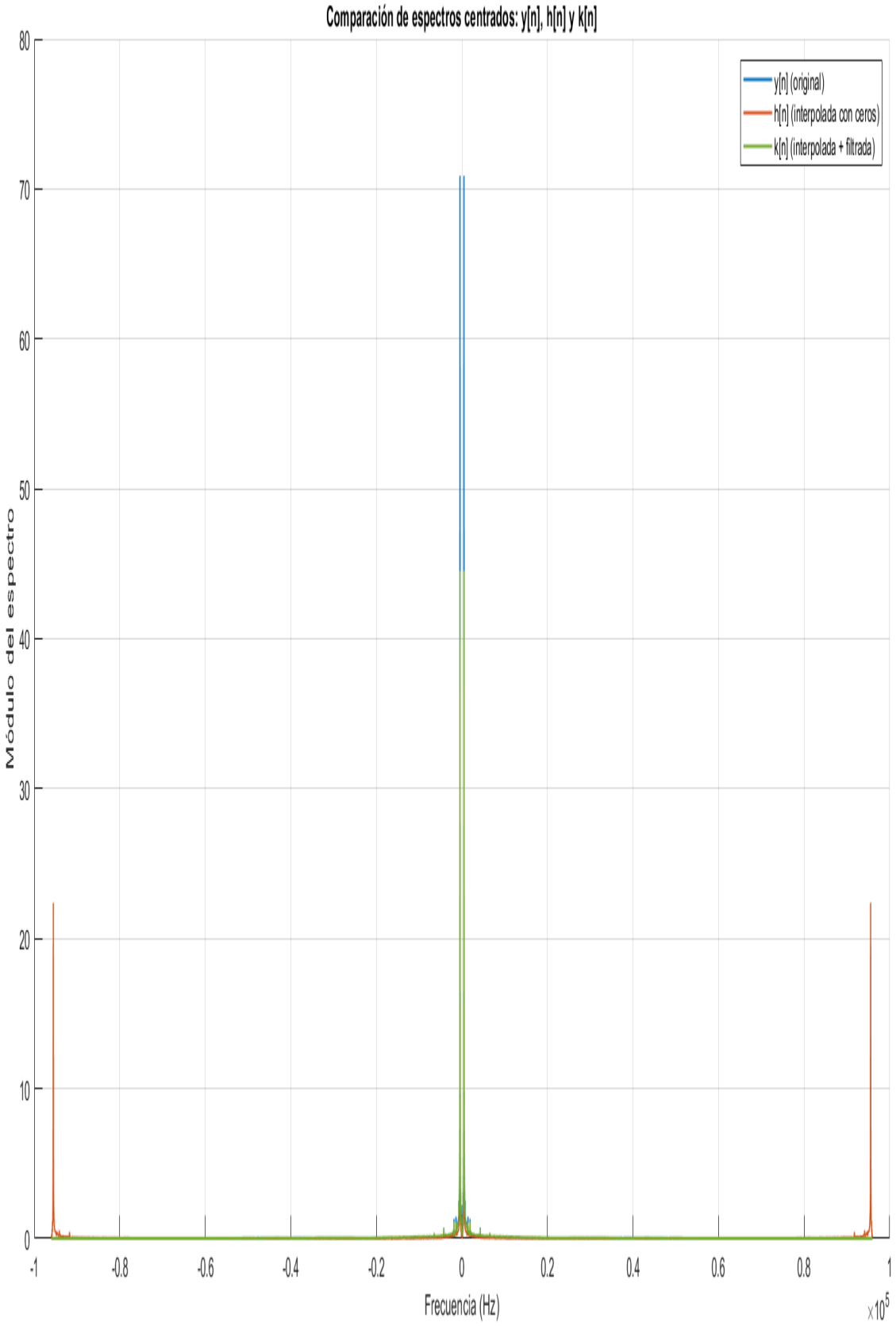




h) Justifique gráficamente, en el dominio de la frecuencia, que los procesos realizados son correctos. No utilice frecuencias normalizadas (frecuencia digital).

```
Nfft = 4096;  % o una potencia de 2 mayor que length(y)

Y = fftshift(fft(y, Nfft)./length(y));          % FFT
Hn = fftshift(fft(h_n, Nfft)./length(h_n));
K  = fftshift(fft(k,  Nfft)./length(k));

% Eje de frecuencias centrado (No normalizadas)

f_y  = (-Nfft/2:Nfft/2-1) * (fs1 / Nfft);      % y[n] → fs1
f_hn = (-Nfft/2:Nfft/2-1) * (fs1*L / Nfft);    % h[n] → fs1*L
f_k  = (-Nfft/2:Nfft/2-1) * (fs1*L / Nfft);    % k[n] → fs1*L

% Representación del módulo
figure;

hold on;

plot(f_y,  abs(Y),  'LineWidth', 1.2, 'Color', [0 0.447 0.741]);   % azul
plot(f_hn, abs(Hn), 'LineWidth', 1.2, 'Color', [0.85 0.325 0.098]); % naranja
plot(f_k,  abs(K),  'LineWidth', 1.2, 'Color', [0.466 0.674 0.188]); % verde

xlabel('Frecuencia (Hz)');
ylabel('Módulo del espectro');
title('Comparación de espectros centrados: y[n], h[n] y k[n]');
legend('y[n] (original)', 'h[n] (interpolada con ceros)', 'k[n] (interpolada +
filtrada)');
grid on;
hold off;
```

Ejecutando el siguiente código, obtenemos la siguiente imagen:



En ella se aprecia cómo la señal interpolada sin filtrar (h[n]) se encuentra comprimida en el eje de frecuencias con respecto al espectro de la señal original (y[n]) y se cuelan las réplicas no deseadas en mi franja de interés. Una vez Interpolada y filtrada: las réplicas que se cuelan desaparecen. Ambas señales, en el centro, son equivalentes en ancho de banda.

i) ¿De qué otra forma se puede obtener el mismo resultado sin emplear un filtro? ¿Cómo se llama a esta técnica?

Si no quisieramos usar un filtro paso bajo, la otra forma de hacerlo sería mediante el uso de una sinc. Es decir, podemos aplicar 'Interpolación ideal banda limitada' (interpolación sinc), que es la alternativa teórica al filtro digital.